

INPF12 Project

Ahmed Soumbara

April 2026

Contents

1	Context	2
2	How tables are represented	2
3	Base Functions	2
3.1	check_table	2
3.2	insert	3
3.3	prod	3
3.4	projection	3
3.5	restrict	4
4	compute_deps	4
5	compute_elementary_deps	5
6	normalization_level	5
6.1	1NF Guarantee	5
6.2	Key Detection	6
6.3	2NF Check	6
6.4	3NF Check	7
7	Limitations	7
7.1	Data-Driven Detection	7
7.2	Transitivity in Key Detection	7
8	Tests	7
9	Conclusion	8

1 Context

We're working with relational tables and essentially implementing a simple SQL engine . The end goal is , given present information , to find functional dependencies of a table and suggest improvements

2 How tables are represented

A table is represented by a record with two fields: **cols** which is a **schema** (a list of column name and type pairs), and **rows** which is a list of rows. Each **row** is a list of **dbvalue** . The connection between a value in a row and its column is purely positional — the value at index i in a row corresponds to the column at index i in the schema.

Example:

```
1  let example_table = { cols = [ ("id",    (TInt,   false));  
2                                ("name", (TText,  true ));  
3                                ("age",   (TInt,   false)) ];  
4      rows = [ [VInt 1; VText "Alice"; VInt 30];  
5                [VInt 2; VText "Bob";   VInt 25] ]  
6  }
```

3 Base Functions

3.1 check_table

checks if argument `tbl` of type `table` is valid or not .

I saw the best way to implement this is to return `true` if argument follows the following rules :

- Rule 1 : Every row has the same number of columns as the schema.
- Rule 2 : Each value matches the column's type
- Rule 3 : NULL only appears in columns with the schema `(_,(_, true))`
- Rule 4: No duplicate Columns names

NOTE : At this stage , i didn't take into account the 5th rule of validity : *NO duplicate rows values.*

3.2 insert

inserts **row** in **tbl.rows** then checks if the resulting table is valid or not .

3.3 prod

takes 2 tables **tb1** **tb2** and returns the cartesian product between them .//

here i made a helper function **f** whose job is taking a row = **cst** and combining it with every row of **bis**

```
1   let rec f cst bis acc =
2   match bis with
3   |[] -> acc
4   |l::ls -> let acc' = [cst@l]@acc in
5             f cst ls acc'
```

and at the end we use `List.concat_map` (which is the same as `List.concat(List.map())`) to apply **f** and flatten the results into a single list instead of a list of lists , we understand this from its description in List module : From <https://ocaml.org/manual/5.4/api/List.html> :

```
1   val concat : 'a list list -> 'a list
```

description : Concatenate a list of lists. The elements of the argument are all concatenated together (in the same order) to give the result. Not tail-recursive (length of the argument + length of the longest sub-list).

```
1   Example :
2   List.concat [[1]; [2; 3]; [4]]
3   (*      [1; 2; 3; 4] *)
```

3.4 projection

The goal is to build a new table by keeping in the columns the ones specified by **field** . the tricky part is making the new rows with the extracted columns , because given the implementation , the only connection between rows and "cols" is the index number , so we need a function that given a column name returns its index number in the **tbl.cols** list so we can use that index to extract the rows corresponding value and build a new row.

that function is named **find_row_index**.

```
1   let rec find_row_index field counter cols =
2   match cols with
3   |[]-> failwith "field_not_found_in_cols"
4   |(name, _)::rest_cols -> if(name = field) then counter
5                             else find_row_index field (counter +1) rest_cols
6   ;;
```

its gonna prove usefull in subsequent implementation

First we filter the columns :

```
1
2   let find_col field = List.find (fun (name,(_,_))->
3                                   name = field) tbl.cols
4   in
5
6   let new_cols = List.map (fun field -> find_col field) fields in
```

Note: find f l returns the first element of the list l that satisfies the predicate f, so it won't work if theres columns with the same name (rule 4 of table validity protects from that) .

then using those column names , we extarct the corresponding indexes and use those to build new rows .

3.5 restrict

to make restrict we just use `List.filter` on `tbl.rows` , meaning we apply filter on the list of list "tbl.rows" to only return the lists "rows" that satisfy the predicate f .

4 compute_deps

To get ALL dependencies first we need 2 functions :

- **subsets** : given a list , this function returns all possible subsets (or sublists) . For example : `["a";"b"; "c"] = [[]](= empty list); ["c"]; ["b"]; ["b"; "c"]; ["a"]; ["a"; "c"]; ["a"; "b"];["a"; "b"; "c"]]`
- **all_pairs** : returns all possible pairs of a given list

with these 2 functions we can check for every `lhs,rhs` belonging to a subset of **col_names** we check if the dependency $lhs \rightarrow rhs$ holds . In general a dependancy $X \rightarrow Y$ is true if knowing the value in column X gives the value of column Y , meaning given a value of X , all values of Y should be the same . We use this logic to make the **check_fd** function that follows the following logic :

given the 2 fields lhs and rhs, first we project on lhs,rhs

then we extract all unique values in lhs

for each unique value v , we restrict the projected table to where lhs = v

if all rhs values are equal return true (the dependency lhs $\rightarrow$ rhs holds)

otherwise false

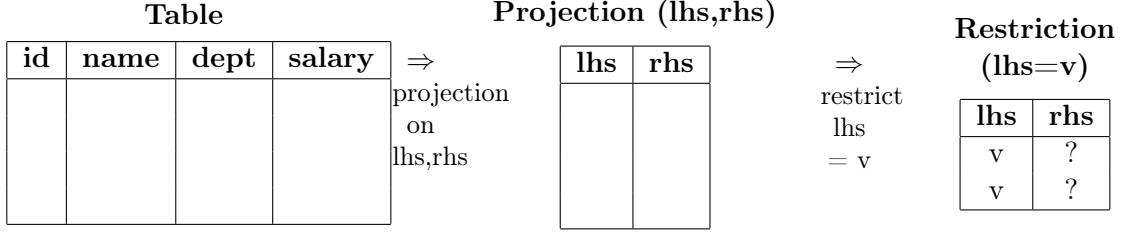


Figure 1: Illustration of `check_fd` for `lhs  $\rightarrow$  rhs`

all that's left is to filter all possible combinations with this `check_fd` function.

but this turns out to give a lot of redundant and duplicate dependencies. So even though the function should return ALL dependencies, I chose to filter them and not show the unions.

for all dependencies of form $X \rightarrow YZ$, if we already have $X \rightarrow Y$ and $X \rightarrow Z$ as dependencies, there's no need to keep the dependency $X \rightarrow YZ$. This is the logic of `check_union`.

5 compute_elementary_deps

calls `compute_deps` to get `all_fds` and filters them to only keep the ones that verify :

- `check_atomic` checks if `fd=(lhs,rhs)` is atomic, meaning if the length of `rhs` is lesser than 2.
- `check_redundancy (lhs,rhs)` is redundant if there exists a `fd (l,r)` in `all_fds` such that `r = rhs` and `l` is a strict subset of `lhs` (see `is_strict_subset` function)

6 normalization_level

6.1 1NF Guarantee

A table is in 1NF if every cell contains an atomic value — meaning a single, indivisible value, not a list. And in our model, 1NF is automatically satisfied by every table, that's because the type `dbvalue` only allows atomic values `VInt`, `VText`, and `VNull`. It is impossible to represent a non-1NF table since there is no `VList` or `VTable` constructor. Therefore `normalization_level` always returns at least 1.

6.2 Key Detection

Before checking 2NF and 3NF, we need to find the keys of the table. A key is a set of columns that determines all other columns.

To find keys, we first merge all functional dependencies that share the same lhs into a single dependency, then check if lhs @ rhs covers all columns:

```
1 (* a lhs is a key if lhs @ rhs covers all columns *)
2 let keys_fds = List.filter (fun (lhs, rhs) ->
3   same_elements (lhs @ rhs) all_cols
4 ) merged_fds
```

We then keep only **minimal** keys, a key is minimal if no strict subset of it is also a key:

```
1 let keys = List.filter (fun key ->
2   not (List.exists (fun other_key ->
3     is_strict_subset other_key key
4   ) all_keys)
5 ) all_keys
```

6.3 2NF Check

A table is in 2NF if it is in 1NF and no non-key attribute is partially dependent on a key, meaning no non-key attribute depends on a *strict subset* of a key.

First we find all non-key attributes, columns that don't belong to any key:

```
1 let att_not_in_keys = List.filter (fun col ->
2   not (List.exists (fun key -> List.mem col key) keys)
3 ) all_cols
```

Then for each non-key attribute, we check if any strict subset of any key determines it:

```
1 (* does any strict subset of this key determine att? *)
2 let key_partially_determines key att =
3   List.exists (fun subkey ->
4     subkey_determines subkey att
5   ) (strict_subsets_of key)
6
7 (* is att partially dependent on ANY key? *)
8 let is_dependent_on_subkey att =
9   List.exists (fun key ->
10     key_partially_determines key att
11   ) keys
```

If any non-key attribute is partially dependent on a key, the table fails 2NF and we return 1.

6.4 3NF Check

A table is in 3NF if it is in 2NF and for every elementary dependency $X \rightarrow A$:

- either X is a key
- or A belongs to some key

```
1 let is_3nf_cond = List.for_all (fun (x, ak) ->
2   match ak with
3   | [a] -> List.mem x keys ||
4     List.exists (fun key -> List.mem a key) keys
5   | _ -> false
6 ) all_elem_fds
```

If any elementary dependency violates this condition, the table fails 3NF and we return 2. Otherwise we return 3.

7 Limitations

7.1 Data-Driven Detection

Functional dependencies are detected purely from the data present in the table. A dependency that doesn't manifest in the current data will not be detected. Conversely, a dependency that holds by coincidence in the current data will be incorrectly reported. For example, if all employees happen to have unique names, $\text{name} \rightarrow \text{id}$ will be detected even though it is not a real constraint. This can cause `normalization_level` to incorrectly classify a table

7.2 Transitivity in Key Detection

Our key detection does not handle transitivity. A key whose determination requires chaining multiple functional dependencies will not be detected. A complete implementation would require a *closure algorithm*, computing the set of all attributes determinable from a given set of attributes by repeatedly applying known dependencies.

8 Tests

to compile test i just ran this code

```
1   ocamlc -g -c projet.ml
2   ocamlc -g -c tests.ml
3   ocamlc -g projet.cmo tests.cmo -o test_projet
4   ./test_projet
```

9 Conclusion

I found this project pretty interesting , it was a good opportunity to get comfortable with some notions i didn't really understand in class , the list module . it was also a good opportunity to recall notions i was already familiar with in CBDR11 .the most challenging part of it was the last 3 functions ,especially *ormalisation_{level}* and *compute_deps* , the most challenging part was time management because this is the first project we have in tandem with studies .contrary to the last PRIM11 project in which we had the whole vacation to focus on it .